```
        policyBuilder.RequireAssertion(context =>                ❶
        {
            var joiningDateClaim = context.User.FindFirst(c =>   ❷
            ➥ c.Type == "Joining Date")?.Value;                 ❷
            var joiningDate = Convert.ToDateTime(                ❷
            ➥ joiningDateClaim);                                ❷
            return context.User.HasClaim("Permission",
            ➥ "View Roles") &&                                  ❸
                joiningDate > DateTime.MinValue &&               ❹
                joiningDate < DateTime.Now.AddMonths(-6);        ❹
    }));
});
```

❶ Use the RequireAssertion method, which takes an AuthorizationHandlerContext as a parameter providing access to the current user.

❷ Use the FindFirst method to access a claim and obtain its value (if there is one) and convert it to a DateTime.

❸ Use the HasClaim method to establish that a claim with the specified value exists.

❹ Compare the joining date value with DateTime.MinValue and the current date to ensure the claim is not null and the date is earlier than six months ago.

The built-in requirements and handlers should cover most common authorization requirements. If you have a lot of complex authorization policies you want to apply to your application, your `Program` class will fill up pretty quickly with multiple assertions. At that point, you can move the code out of authorization configuration and into separate classes by writing your own custom requirements and handlers.

10.3.4 Custom authorization requirements and handlers

Authorization requirement classes implement the `IAuthorizationRequirement` interface. It is an empty marker interface, defining no members. Handlers are represented by the `IAuthorizationHandler` interface, which defines the